

MixBytes()

Lido GGV Migrator V2 Security Audit Report

MAY 27, 2026

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	4
1.4 Security Audit Methodology	6
Stage 1: Interim Audit	6
Stage 2: Re-audit & Mainnet Deployment Verification	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	11
2.1 Critical	11
2.2 High	11
2.3 Medium	11
2.4 Low	11
L-1 MAX_ALLOWED_CUMULATIVE_ERROR is enforced per call, not across partial migrations	11
3. About MixBytes	12

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`GGVMigratorV2` is the V2 of the operator-driven, one-shot migrator audited previously (commit `da932764a3e9fbc71127f39aa1006a2ddf73d46c`, deployed at `0x00000000433bE90C51EE86E801C65977cFb42d9F`). It moves a leveraged Aave V3 position (`weETH` collateral / `variableDebtEthWETH` debt) from Lido's Golden Goose Vault (GGV, `0xef417FCE1883c6653E7dC6AF7c6F85CCDE84Aa09`) into the Mellow `strETH` subvault at `0x893aa69FBAA1ee81B536f0FbE3A3453e86290080`. All hardcoded addresses (Aave V3 Pool, Aave Oracle, `aTokens`, `WETH`, `weETH`, GGV, `strETH` subvault) are unchanged from V1 and match mainnet.

The codebase was audited via diff review against the V1 audit baseline (commit `da932764a3e9fbc71127f39aa1006a2ddf73d46c`) and manual review of the new partial-migration logic on commit `07adcc04ecc3279edb9de0b67194ab0a91dbe025`.

The V2 diff against V1 is narrow and additive. The single new capability is partial migration: `migrate()` takes a new `percentageD6` parameter in `[0, 1e6]` that controls what fraction of GGV's current `variableDebtEthWETH` balance is retired in this call, allowing the operator to spread the migration across multiple transactions instead of clearing the full debt atomically. Supporting changes: `calculateSteps` sizes iterations against `wethToMigrate = wethDebt * percentageD6 / 1e6` and returns an `expectedWethAfter` that the post-loop residual-debt check enforces as a two-sided bound (`|ggvWethDebt - expectedWethAfter| <= MAX_WETH_DEBT_ERROR`, replacing V1's one-sided ceiling); the `migrated` flag is set only when the call clears the debt below `MAX_WETH_DEBT_ERROR`, allowing successive partial calls; `ReentrancyGuard` is added to `migrate()` and V1's "set `migrated=true` before the loop" defense is dropped (the flag now reflects completion state, not call state); a new `PartiallyMigrated(timestamp, caller, cumulativeError, percentageD6)` event distinguishes intermediate calls from the final `Migrated`. Constants `MIN_WETH_DEBT_BEFORE_MIGRATION` and `MAX_WETH_DEBT_AFTER_MIGRATION` were renamed (to `MIN_WETH_DEBT_FOR_MIGRATION` and `MAX_WETH_DEBT_ERROR`) but values are unchanged. All other invariants - HF floors at 1.01 on both GGV and the subvault, the 30-iteration cap, the \$1 cumulative-equity-drift bound per call, owner-only access, the `wethToBorrow <= wethToRepay` guard, and the deterministic loop body - are preserved from V1.

One operator-facing constraint introduced by the new sizing logic is worth surfacing in the documentation rather than as a finding. The `InsufficientWethDebt` check at lines 59-64 forbids leaving `expectedWethAfter` in the forbidden range (`MAX_WETH_DEBT_ERROR`, `MIN_WETH_DEBT_FOR_MIGRATION`) = `(1 gwei, 1 ether)`. Consequence: once GGV's WETH debt is below 2

ether, the next `migrate()` must finish the job. Operators should size early partials so each remainder stays ≥ 2 ether until the closing call.

One Low-severity finding was raised and no Critical, High, or Medium issues were identified.

Out of scope for this audit: Aave V3 protocol, the Mellow flexible-vaults framework (subvault, CallModule, VerifierModule), the Veda BoringVault implementation, and any off-chain components (operator tooling, merkle-root generation, keepers).

1.3 Project Overview

Summary

Title	Description
Client	Lido
Category	Liquid Staking
Project	GGV Migrator V2
Type	Solidity
Platform	EVM
Timeline	26.05.2026 - 27.05.2026

Scope of Audit

File	Link
src/utils/GGVMigratorV2.sol	src/utils/GGVMigratorV2.sol

Versions Log

Date	Commit Hash	Note
26.05.2026	07adcc04ecc3279edb9de0b67194ab0a91dbe025	Initial Commit

Mainnet Deployments

File	Address
GGVMigratorV2.sol	0x00000000333B254A17361C550ec49cED9D40E0F4

The deployment was reviewed and its configuration matches the audited scope. Deployed bytecode is compiled from the [07adcc04ecc3279edb9de0b67194ab0a91dbe025](#) commit and the hardcoded `SUBVAULT` constant resolves to [0x893aa69FBAA1ee81B536f0FbE3A3453e86290080](#) as expected. The owning Safe ([0x81698f87C6482bF1ce9bFcFC0F103C4A0Adf0Af0](#)) is the same Safe that owned the V1 deployment and uses the canonical Gnosis Safe v1.3.0 singleton ([0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552](#)), threshold `5`, `8` owners (all EOAs, no nested Safes or controller contracts). Compromise of the owning Safe is unlikely under this configuration (5-of-8 threshold across independent EOA signers, no modules to bypass it, and canonical Safe contracts).

1.4 Security Audit Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:

- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:

- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:

- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50-60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	0
Low	1

Findings Statuses

ID	Finding	Severity	Status
L-1	<code>MAX_ALLOWED_CUMULATIVE_ERROR</code> is enforced per call, not across partial migrations	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

Not Found

2.4 Low

L-1	MAX_ALLOWED_CUMULATIVE_ERROR is enforced per call, not across partial migrations		
Severity	Low	Status	Acknowledged

Description

In `GGVMigratorV2.sol` at lines 160-171, `migrate()` measures equity drift on GGV and the subvault before and after the loop, sums the two absolute deltas, and reverts if the total exceeds `MAX_ALLOWED_CUMULATIVE_ERROR = 1e8` (~\$1 in Aave's 8-decimal base). V1 ran this check exactly once per migration because `migrate()` was one-shot. V2 introduces partial migration via `percentageD6`: the operator can call `migrate()` repeatedly with `migrated` staying `false` until the final call clears GGV's debt below `MAX_WETH_DEBT_ERROR`. The cumulative-error check still uses the same per-call bound, so each call independently consumes up to ~\$1 of allowed drift. Across N partial calls the worst-case drift on the combined (GGV + subvault) equity is bounded by $N * 1e8$, not $1e8$. With the 30-iteration cap, the practical ceiling is small - a few dollars of drift in the worst case. No user-funds risk; the per-call HF floors at 1.01 still prevent any partial from leaving either position near liquidation.

Recommendation

We recommend tracking cumulative drift in storage across partial `migrate()` calls and enforcing a single global `MAX_ALLOWED_CUMULATIVE_ERROR` bound, so the total drift across all partials is $\leq 1e8$ rather than $N * 1e8$.

Client's Commentary:

Acknowledged

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle – from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io